SAP Integration Suite

Cloud Integration - Palette, Events, Steps and Adapter Properties explained

What every item does - why it exists - how it works - with diagrams



Inside this guide

- What SAP Integration Suite and Cloud Integration are, and how an integration flow (iFlow) runs
- Every palette group: Participant, Process, Events, Mapping, Transformation, Call, Message Routing, Security Elements, Persistence, Validator and Toolbar
- For each item: the definition, why you use it and how it works
- The bottom-panel properties of the Sender and Receiver adapters, field by field
- Diagrams for the exchange, events, routing, security, persistence, proxy, CSRF, retry and error handling

Contents

Contents 2

1. Overview: SAP Integration Suite 4

1.1 Capabilities of Integration Suite . . . 4
1.2 Key terms . . . 4
1.3 How an iFlow looks and runs . . . 4
1.4 The exchange: headers, properties and body . . . 5
1.5 The palette at a glance . . . 5

2. Participant 7

3. Process 8

4. Events 9

5. Mapping 10

6. Transformation 11

6.1 Content Modifier . . . 11
6.2 Converters . . . 11
6.3 Decoder . . . 12
6.4 Encoder . . . 12
6.5 EDI Extractor . . . 12
6.6 Filter . . . 12
6.7 Message Digest . . . 13
6.8 Script (Groovy and JavaScript) . . . 13

7. Call 14

7.1 External Call . . . 14
7.2 Local Call . . . 14

8. Message Routing 15

8.1 Router . . . 15
8.2 Join . . . 15
8.3 Multicast . . . 15
8.4 Gather . . . 15
8.5 Aggregator . . . 16
8.6 Splitters . . . 16

9. Security Elements 17

10. Persistence 18

10.1 Data Store Operations . . . 18

10.2 Persist . . . 18

11. Validator 19

12. Toolbar actions 20

13. Bottom properties (channel configuration) 21

13.1 Sender adapter (HTTPS example) . . . 21

13.2 Receiver adapter (OData example) . . . 21

14. iFlow end to end: Start, Sender, End, Receiver 26

15. Quick reference and best practices 27

15.1 Which step for which need? . . . 27

15.2 Best practices . . . 27

15.3 Common errors and where to look . . . 27

1. Overview: SAP Integration Suite

SAP Integration Suite is SAP's integration platform as a service (iPaaS) on SAP Business Technology Platform (BTP). It connects cloud and on-premise applications, partners and APIs so that data can move between them reliably, securely and in the right format. The part this guide is about is **Cloud Integration**, the capability in which you design, deploy and monitor **integration flows (iFlows)**. The palette items, events and properties you listed all belong to the iFlow editor of Cloud Integration.

1.1 Capabilities of Integration Suite

Capability	What it does	Why you would use it
Cloud Integration	Designs and runs iFlows: message processing, mapping, routing, adapters	Process-level integration between SAP and non-SAP systems (this guide)
API Management	Publishes, secures and monitors APIs (policies, quotas, developer portal)	Expose your services as managed APIs
Open Connectors	Ready-made connectors to many third-party SaaS applications	Connect to apps without writing adapter code
Integration Advisor	Helps create and maintain B2B interfaces and mappings (MIGs and MAGs)	Speeds up EDI and B2B mapping work
Trading Partner Management	Manages partners, agreements and B2B profiles	Onboard business partners in a structured way
Migration Assistant	Helps move content from SAP Process Orchestration to Cloud Integration	Move older PI/PO scenarios to the cloud
Event capabilities	Event-driven messaging (for example Event Mesh)	Decouple systems through events

Availability of individual capabilities depends on your licence and tenant plan.

1.2 Key terms

Term	Meaning
Tenant	Your own isolated Integration Suite environment, with a design-time part (Web UI) and a runtime part (the engine that runs deployed iFlows).
Integration Package	A container that groups related artifacts (iFlows, value mappings, scripts, APIs).
Integration Flow (iFlow)	The graphical model of one integration scenario: sender, steps, receiver. Built from the palette, deployed to the runtime.
Adapter	The connectivity component attached to a sender or receiver channel (HTTPS, SOAP, OData, SFTP, Mail, IDoc, JMS and more). It handles protocol, address and authentication.
Channel (message flow)	The line that connects a participant (Sender or Receiver) to the iFlow. The adapter is configured on this channel.
Monitoring	Where deployed iFlows and message runs are inspected (status Completed, Failed, Retry, Escalated), including logs and persisted payloads.
Security Material / Keystore	Area where credentials, certificates and keys are stored and referred to by alias.

1.3 How an iFlow looks and runs

A sender sends a message to the endpoint of the iFlow. The sender adapter receives it, the **Start** event begins the **Integration Process**, each step you placed runs in order, and the **End** event hands the result to the receiver adapter. If a step fails, the **Exception Subprocess** (when you have modelled one) takes over.

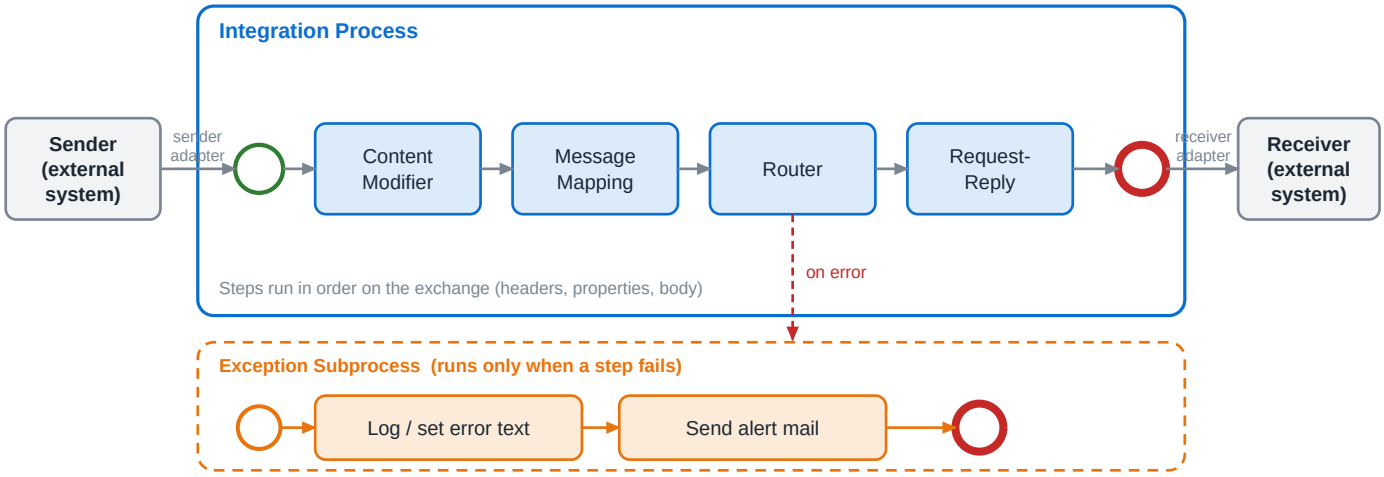


Figure 1 - Anatomy of an iFlow: Sender, Integration Process with steps, Receiver, and an Exception Subprocess.

1.4 The exchange: headers, properties and body

Every step works on one message run, which is held in a **Camel exchange**. Understanding the three parts of the exchange explains almost every step in the palette.

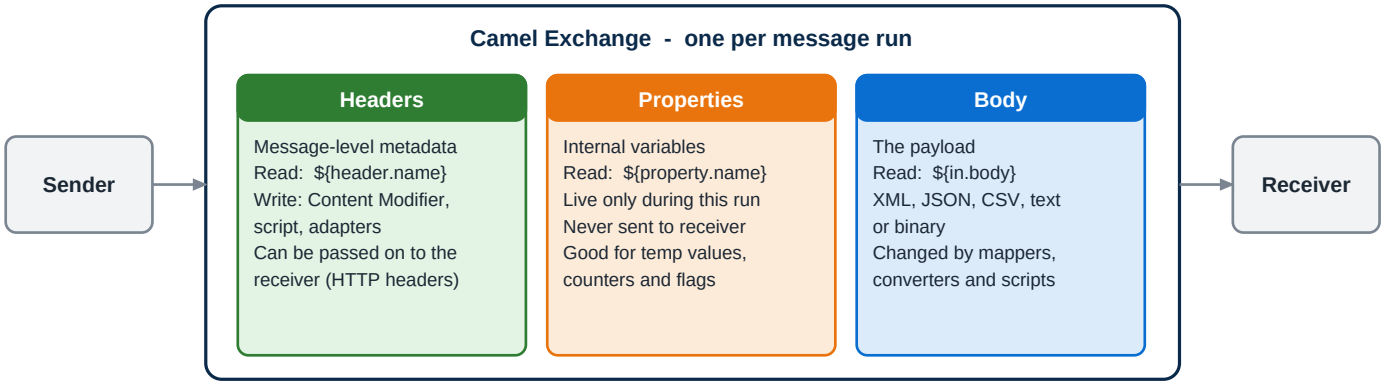


Figure 2 - The three parts of the exchange that every step reads and changes.

Part	Definition	Typical use	Expression to read it
Header	Metadata that belongs to the message	Pass values to the receiver (for example HTTP headers); remember IDs for routing	${header.name}
Property	Internal variable, exists only while the iFlow runs	Keep temporary values, counters, flags, original payload	${property.name}
Body	The payload itself	The XML / JSON / CSV that mappings and converters work on	${in.body}

1.5 The palette at a glance

The palette on the right of the iFlow editor groups the available steps. Figure 3 shows all groups from your list. The following chapters explain every item.



Figure 3 - The complete palette of the iFlow editor, grouped as in your list.

2. Participant

A **participant** is an external party that talks to the iFlow. The iFlow itself (the Integration Process) is not a participant; participants sit outside it and are linked by channels.

Item	Definition	Why / how it works
Sender	The system or partner that sends the message into the iFlow.	You connect it to the Start Message event with a channel. In that channel you choose the **sender adapter** (HTTPS, SOAP, SFTP, IDoc, JMS, Mail and others) and configure address, authorization and limits (see chapter 13.1). The sender is what triggers the iFlow, except for timer-started flows.
Receiver	The system that gets the final (or an intermediate) message.	You connect it from an End Message event, or from a Request-Reply / Send / Content Enricher step. In the channel you choose the **receiver adapter** (OData, SOAP, HTTP, SFTP, Mail, IDoc and others) and configure address, proxy type, authentication, operation and retry (see chapter 13.2). You can have many receivers in one iFlow.

Tip: keep the address, credentials and other environment-specific values as **externalized parameters** so that the same iFlow can be transported from test to production without editing.

3. Process

A **process** is a container that holds a sequence of steps. Three kinds exist in the palette.

Item	Definition	Why / how it works
Integration Process	The main process of the iFlow. It is where the message is received and where the normal flow runs.	Every iFlow has one. Execution begins at its start event and ends at an end event. All other processes are called from it or triggered by errors in it.
Local Integration Process	A reusable sub-process inside the same iFlow.	It is started by a **Process Call** (once) or a **Looping Process Call** (repeat while a condition is true) from the main process. It avoids duplicating steps and lets you build loops, for example paging through an API until no data is left. It begins with a plain Start Event and ends with an End Event or an Escalation End Event.
Exception Subprocess	A handler inside a process that runs when a step raises an error.	It starts with an **Error Start Event**. Inside you can log the error, build a custom error message, send an alert e-mail, store the payload, and then finish with an **End** (error handled, flow ends normally) or **Error End Event** (error is passed on and the run is marked failed). The caught exception is available as the property `CamelExceptionCaught`.

Main Integration Process

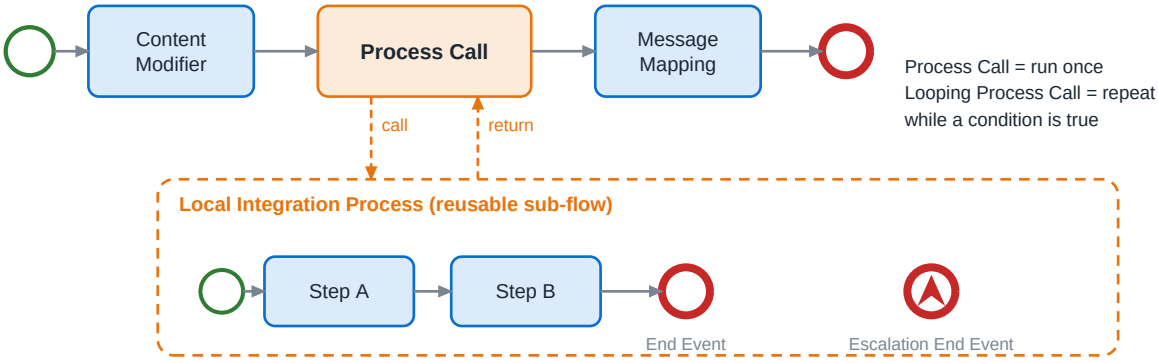


Figure 4 - Main process calling a Local Integration Process (call and return).

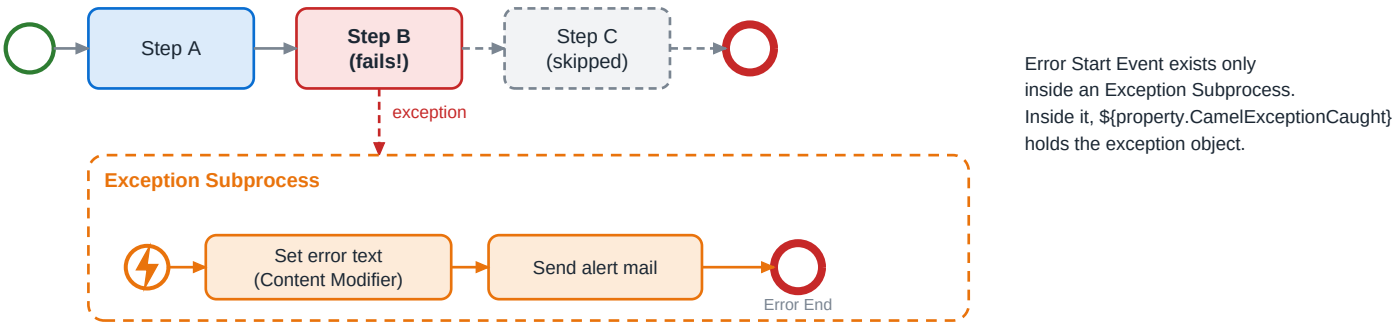


Figure 5 - How an Exception Subprocess takes over when a step fails.

4. Events

An **event** is something that happens at the start or end of a process. Events are drawn as circles: a thin ring starts a process, a thick ring ends it.

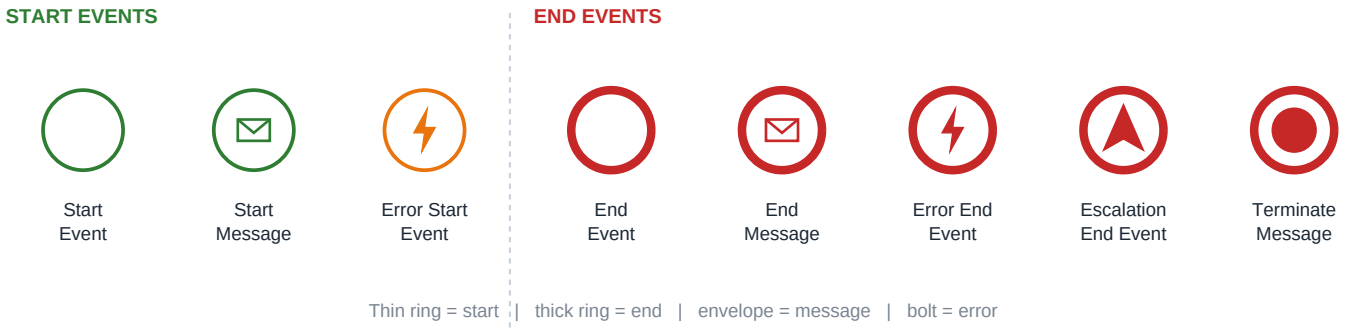


Figure 6 - Event symbols in the palette.

Event	Definition	Why / how it works	Typical use
Start Event	Plain start of a process.	Begins a process that is not started by a message from a sender. The start of a Local Integration Process is a plain Start Event.	Beginning of a Local Integration Process
Start Message	Start of a process triggered by an incoming message.	The channel from a Sender ends here. When the sender adapter receives a message, the process starts with that message as its body.	Main process started by a sender
Error Start Event	Start of an Exception Subprocess.	Fires when a step in the surrounding process throws an exception. It can only be used inside an Exception Subprocess.	Error handling
End Event	Plain end of a process.	Finishes the process without sending the message to a receiver. In a synchronous scenario the final message is returned to the sender. Used also at the end of local processes and exception subprocesses.	Normal completion, local process end
End Message	End that sends the message to a receiver.	The message is delivered through the channel and adapter attached to this event. Mark the point where the iFlow's result leaves toward a target system.	Last step is delivery to a receiver
Error End Event	End that raises an error.	Stops the run and reports an error. The message status becomes Failed; in synchronous scenarios the sender receives an error response. Used to make an error visible after handling it.	Return a custom error to the sender
Escalation End Event	End of a Local Integration Process that raises an escalation.	The escalation is passed to the calling process, which treats it as an error and can handle it in its own Exception Subprocess.	Signal a problem from a local process to the main process
Terminate Message	Immediate stop of message processing.	Ends processing at once; no further steps run in this branch. Often used in a Router branch for messages that need no more processing (for example filtered-out data).	Drop messages that should not go on

Note: a **Timer** start event also exists in the palette for scheduled runs (it is not in your list, so it is not detailed here).

5. Mapping

Mapping steps change the structure or values of a message so that the sender's format fits what the receiver expects.

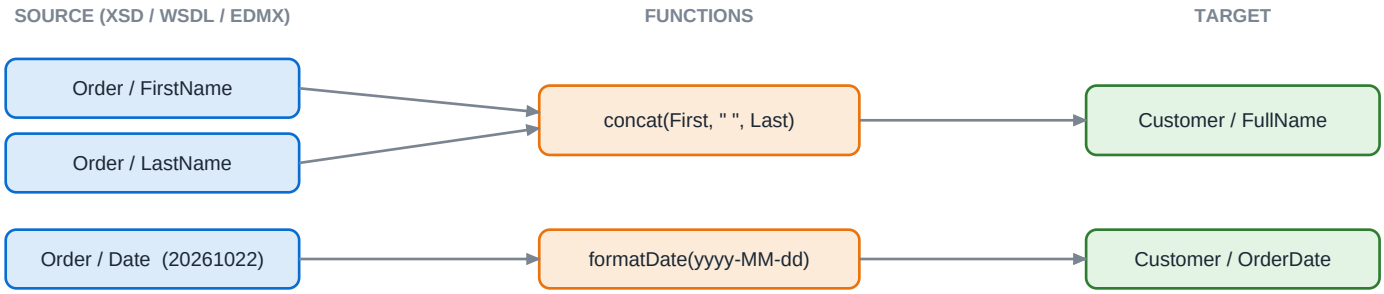


Figure 7 - Message Mapping: source fields, functions and target fields.

Item	Definition	Why / how it works
Message Mapping	Graphical mapping of source structure to target structure.	You upload or choose a source and target structure (XSD, WSDL or EDMX) and draw lines between fields. Built-in functions cover text (concat, substring), arithmetic, date (formatDate), boolean (if, ifElse) and node functions (splitByValue, collapseContexts). Custom functions can be added with Groovy or JavaScript. It is the most common mapping.
ID Mapping	Maps identifiers between systems, for example sender ID to receiver ID.	The pair of IDs is stored by the tenant, so the same source ID always maps to the same target ID for the same object. Useful for master data where the two systems use different keys.
Operation Mapping	Maps a whole service operation (request and response messages).	Used when the integration is based on service interfaces (WSDL operations). It combines the mapping of the request and of the response and is typically created for interface-driven scenarios, for example with Integration Advisor or Process Orchestration content.
XSLT Mapping	Transformation by an XSLT stylesheet.	You upload an XSLT file; the engine applies it to the XML body. Best for complex restructuring that is awkward in graphical mapping. Parameters can be passed from headers or properties.

6. Transformation

Transformation steps change the message content, format or encoding without being a graphical mapping.

6.1 Content Modifier

Definition: a step that sets or changes the message **headers**, the **exchange properties** and the **body**. **Why:** almost every iFlow needs values that are read from one place and used in another. **How:** it has three tabs; in each you add rows, each with a name, a source type (Constant, Header, Property, XPath, Expression, Local or Global Variable, External Parameter) and a value.

Tab	What it changes	Example row
Message Header	Headers that can travel on to the receiver	Name: Content-Type, Source Type: Constant, Value: application/json
Exchange Property	Internal variables for later steps	Name: orderId, Source Type: XPath, Value: /Order/ID, Data Type: java.lang.String
Message Body	The payload (replaces it)	Type: Expression, Body: a template that uses ${header.x} and ${property.y}

Expression	Result
${header.name}	Value of the header "name"
${property.name}	Value of the exchange property "name"
${in.body}	The current body
${date:now:yyyy-MM-dd}	Current date in the given format
${camelId}	Identifier of the Camel context (useful for logging)

6.2 Converters

A **converter** changes the data format of the body. Because mappers, filters, splitters and XPath work on XML, you normally convert to XML first and back to the target format last.



Figure 8 - Converters use XML as the central working format.

Converter	Definition	Why / how it works
CSV to XML Converter	Turns delimited text into XML.	You give the XML element name and namespace, the field separator and whether the first line is a header. Each line becomes a record element with one child per field.
XML to CSV Converter	Turns XML into delimited text.	You give the XPath of the repeating element and the field separator; each repeating element becomes one line.
EDI to XML Converter	Turns an EDI interchange (UN/EDIFACT, ANSI X12 and others) into XML.	Uses the EDI schema to parse segments and elements. Belongs to the B2B capabilities and needs the matching licence.
XML to EDI Converter	Builds an EDI interchange from XML.	Uses the schema to create segments and separators; used for the outbound B2B side.

Converter	Definition	Why / how it works
JSON to XML Converter	Turns a JSON document into XML.	JSON has no root element, so you can add one. Names that are not valid XML names can be replaced. Arrays become repeated elements.
XML to JSON Converter	Turns XML into JSON.	Often used just before sending to a REST or OData receiver. Options control the root element and how numbers, booleans and arrays are written.

6.3 Decoder

A **decoder** reverses an encoding or compression so that the content becomes usable again.

Item	Definition	Why / how it works
Base64 Decoder	Converts Base64 text back to binary or plain content.	Base64 is used to carry binary data (for example attachments) in text formats like XML or JSON. Decode it to read or process the real file.
GZIP Decompression	Unpacks one GZIP-compressed payload.	Compressed messages are smaller and faster to transfer. Decompress before mapping or converting.
MIME Multipart Decoder	Splits a MIME multipart message into its parts.	The first part becomes the body and the other parts become attachments. Headers can optionally be kept.
ZIP Decompression	Unpacks a ZIP archive.	The archive entries are made available one after another; combine with a Splitter if you need to handle each file separately.

6.4 Encoder

An **encoder** does the opposite: it encodes or compresses content for transport or storage.

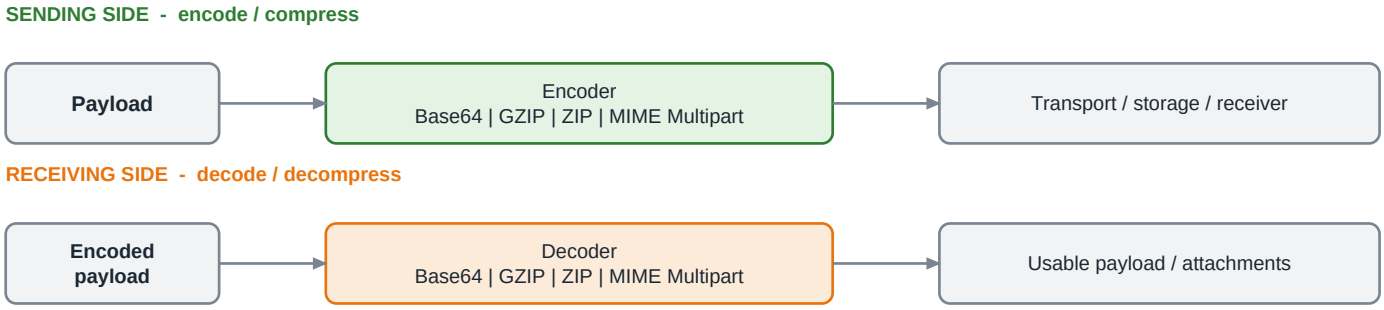


Figure 9 - Encoder on the sending side and decoder on the receiving side.

Item	Definition	Why / how it works
Base64 Encoder	Converts binary or text into Base64 text.	Lets you embed binary content in XML or JSON. The size grows by about one third.
GZIP Compression	Compresses the payload as one GZIP stream.	Reduces size for large messages and slow links. The receiver must be able to decompress.
MIME Multipart Encoder	Packs body and attachments into one multipart message.	You choose the multipart subtype and whether headers are included. Used for HTTP and mail scenarios with attachments.
ZIP Compression	Creates a ZIP archive with the content.	Used when the receiver expects a ZIP file or when several files are sent together.

6.5 EDI Extractor

Definition: a step that reads the header information of an EDI document (for example the sender and receiver identifiers, the document type and the interchange number) and writes it to message headers or properties. **Why:** routing and validation need those values. **How:** it parses the envelope of the EDI interchange (UN/EDIFACT or ANSI X12) and exposes the values, so a Router can send the message to the right branch based on partner and document type. It belongs to the B2B capabilities.

6.6 Filter

Definition: a step that keeps only the part of an XML message you select with an **XPath** expression and removes everything else. **Why:** reduces the message to what is needed, or removes envelopes. **How:** enter the XPath of the node to keep, and choose the value type (Nodelist or String). Nothing is returned when the XPath finds nothing, so use it with care.

6.7 Message Digest

Definition: a step that calculates a **hash value** (digest) of the message or of selected nodes and stores it in a header. **Why:** to prove integrity, detect duplicates or build an idempotency key. **How:** choose the algorithm (for example SHA-256), optionally canonicalize the XML and select the parts to include; the result is placed in the header you name.

6.8 Script (Groovy and JavaScript)

Definition: steps for custom logic written in code. **Why:** when no standard step does what you need, for example complex string handling, loops, or calling a library. **How:** you write a function `processData` that receives the `Message` object and returns it. Through that object you read and write body, headers and properties.

Item	Language	Notes
Groovy Script	Groovy (runs on the JVM)	Most commonly used; has access to standard Java classes and the Cloud Integration script API. Use for JSON/XML parsing and loops.
JavaScript	JavaScript (Rhino-based)	Same message API; suitable for simple logic. Not all modern JavaScript features are available.

```
// Groovy
import com.sap.gateway.ip.core.customdev.util.Message
def Message processData(Message message) {
    def body = message.getBody(String)          // read body
    def id   = message.getHeader("OrderId", String)
    message.setHeader("MyHeader", "value")      // write header
    message.setProperty("myProp", "x")          // write property
    message.setBody(body.toUpperCase())         // write body
    return message
}
```

```
// JavaScript
importClass(com.sap.gateway.ip.core.customdev.util.Message);
function processData(message) {
    var body = message.getBody();
    message.setHeader("MyHeader", "value");
    message.setProperty("myProp", "x");
    return message;
}
```

7. Call

Call steps leave the current flow temporarily: either to an external system, or to a local sub-process.

7.1 External Call

An external call uses a **receiver channel** (so an adapter) in the middle of the flow. Four variants exist.

Variant	Definition	Why / how it works
Request-Reply	Sends a request and waits for the response.	The response replaces the current body. Used to fetch data or confirm a result (for example read a master record or create an entity and get its key).
Content Enricher	Calls a service and merges the response into the current message.	You choose how the original and the looked-up message are combined (by key path and node). Useful to add details to each record.
Poll Enrich	Fetches a message or file from a source on demand.	Reads from a polling adapter (SFTP, mail, JMS and others) inside the flow, with a timeout, and merges it with the current message.
Send	Sends the message to a receiver and continues without using a response.	Fire and forget; for example write an archive copy or notify another system, while the main flow continues.

7.2 Local Call

Variant	Definition	Why / how it works
Process Call	Runs a Local Integration Process once.	Choose the local process by name. The message goes in, is processed, and returns to the main process.
Looping Process Call	Runs a Local Integration Process repeatedly.	Set a loop condition (an expression) and a maximum number of iterations. Used for paging through large result sets or retry-until patterns.

8. Message Routing

Routing steps decide where a message goes, copy it to several paths, split it into pieces, or bring pieces together.

8.1 Router

Definition: a step that sends the message into **one** of several branches, based on conditions. It works like if / else-if / else. **How:** each route has an **expression** (XML condition with XPath, or non-XML condition such as `${header.type} = 'A'`) and an order; the first true route is taken. Exactly one route must be marked as **Default Route** that handles everything else. Each branch ends at its own end event or at a **Join**.

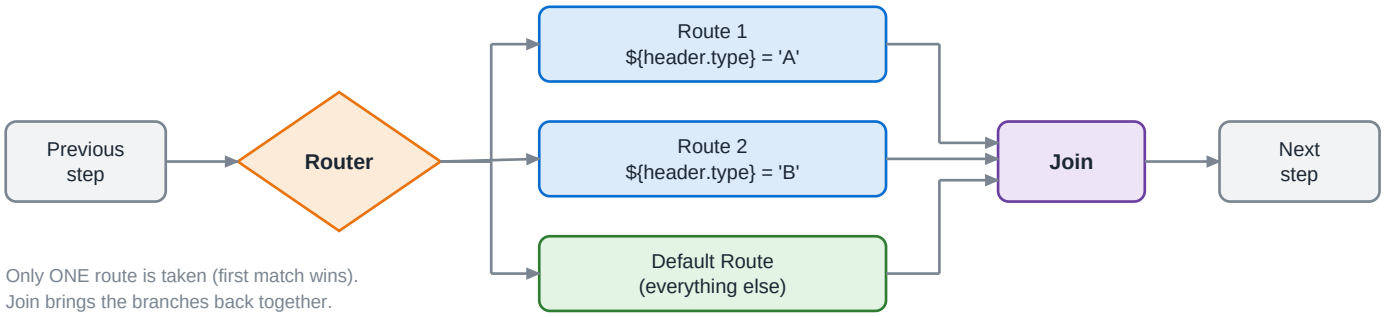


Figure 10 - Router with two conditional routes and one default route, merged by a Join.

8.2 Join

Definition: a step that merges branches created by a Router or a Multicast back into a single path. **Why:** so that the following steps are written only once. It does not combine message content; use **Gather** for that.

8.3 Multicast

Item	Definition	Why / how it works
Parallel Multicast	Sends a copy of the message to every branch at the same time.	Faster when the branches are independent. The order of completion is not fixed.
Sequential Multicast	Sends a copy to the branches one after another.	Use when order matters or when a later branch depends on an earlier one. You set the branch order and whether to stop on exception.

Parallel Multicast: all branches start together. Sequential Multicast: branch 1, then 2, then 3 (order is configured).

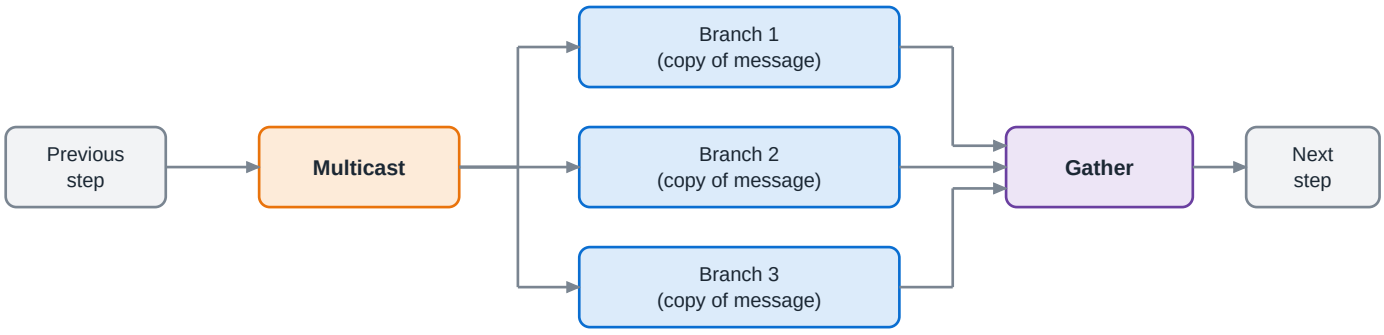


Figure 11 - Multicast copies the message to several branches; Gather merges the results.

8.4 Gather

Definition: a step that merges the messages of several branches (usually of a multicast) into one message. **How:** choose the incoming format (Plain Text, XML with the same format, or XML with different formats) and the aggregation algorithm (for example combine, or combine in sequence). The result is one message again.

8.5 Aggregator

Definition: a step that collects **several related messages that arrive at different times** and sends them on as one. **How:** a **correlation expression** (XPath) tells which messages belong together; an **aggregation strategy** says how they are combined; a **completion condition** (last-message marker, number of messages or timeout) tells when the group is complete. This differs from Gather, which merges branches within one run.

8.6 Splitters

A **splitter** breaks one message into several smaller messages that are handled one by one (sequentially or in parallel). A Gather or Join normally follows.

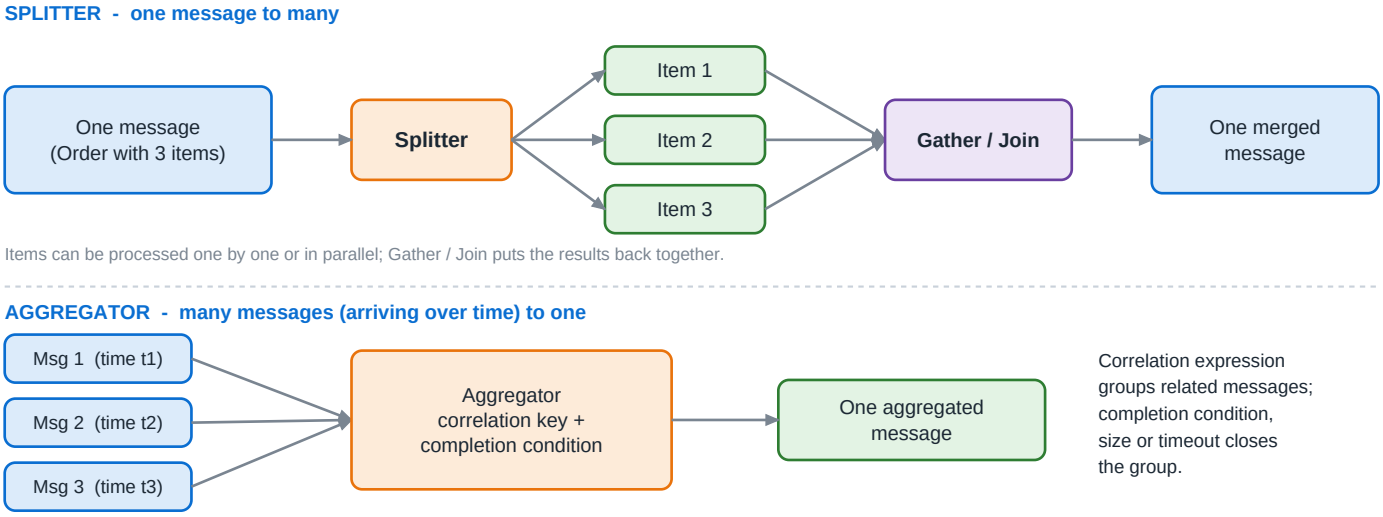
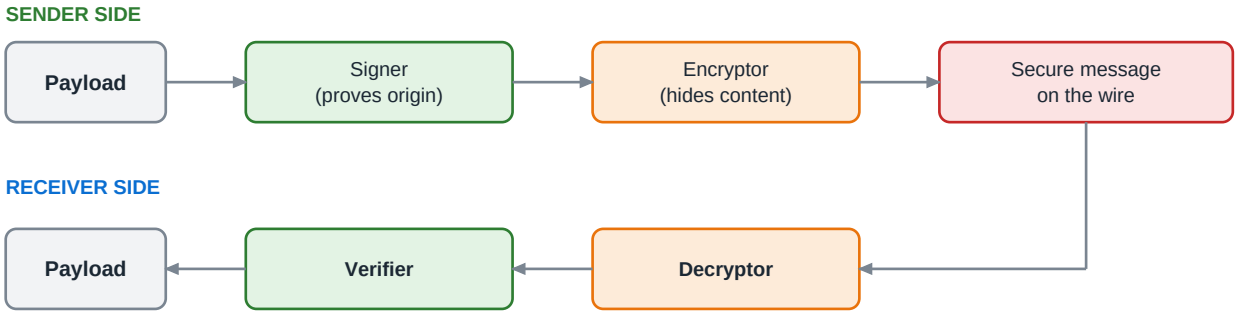


Figure 12 - Splitter / Gather (one to many and back) and Aggregator (many over time to one).

Splitter	Definition	Why / how it works
General Splitter	Splits a composite message into individual messages and keeps the envelope.	Uses an XPath (or line break for text). Each part is wrapped with the original envelope; options include grouping, parallel processing, streaming and stop on exception.
Iterating Splitter	Splits a composite message by a tag or token without the envelope.	Each part is only the element you split on. Use when the envelope is not needed downstream.
EDI Splitter	Splits an EDI interchange into single messages.	Splits by interchange, functional group or message and can validate and acknowledge. Part of the B2B capabilities.
IDoc Splitter	Splits an IDoc bundle into single IDocs.	Each IDoc becomes its own message so that errors and processing are per IDoc.
PKCS#7/CMS Splitter	Splits a PKCS#7/CMS signed or encrypted message into its components.	Separates content from signature (or the enveloped parts) so they can be verified or decrypted individually.
Tar Splitter	Splits a TAR archive into its files.	Each file becomes one message; combine with Content Modifier for file names.
Zip Splitter	Splits a ZIP archive into its files.	Each file entry becomes one message; the entry name is available in a header.

9. Security Elements

Security elements protect messages with **encryption** (confidentiality) and **digital signatures** (origin and integrity). Keys and certificates are referred to by alias and stored in the tenant Keystore / Security Material.



Sign with MY private key; the partner verifies with my PUBLIC key.
Encrypt with the receiver's PUBLIC key; the receiver decrypts with its PRIVATE key.
Keys and certificates live in the tenant Keystore (Security Material).

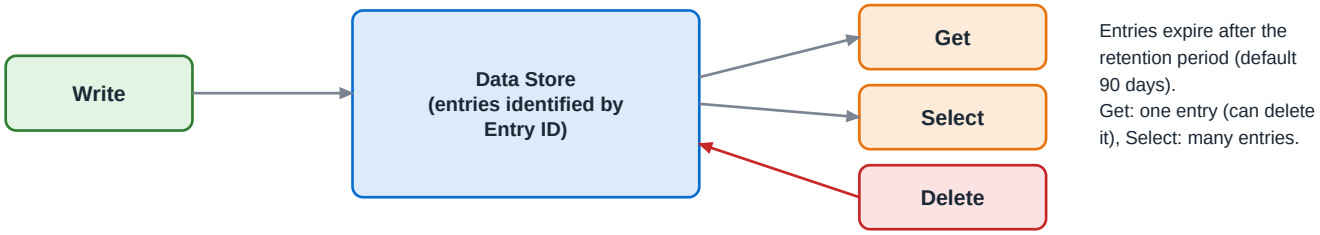
Figure 13 - Signing and encrypting on the sender side; decrypting and verifying on the receiver side.

Group	Item	Definition	Why / how it works
Encryptor	PGP Encryptor	Encrypts the payload using OpenPGP.	Uses the receiver's public PGP key (public keyring); can optionally sign as well. Used for partners who exchange PGP files.
Encryptor	PKCS7 Encryptor	Encrypts the payload as PKCS#7 / CMS enveloped data.	You choose the receiver's public key alias and a content encryption algorithm. Only the holder of the private key can read it.
Decryptor	PGP Decryptor	Decrypts OpenPGP-encrypted content.	Uses the private key in the PGP secret keyring; can also verify an included signature.
Decryptor	PKCS7 Decryptor	Decrypts PKCS#7 / CMS enveloped content.	Uses your private key from the keystore (referenced by alias).
Signer	PKCS7 Signer	Creates a PKCS#7 / CMS digital signature.	Uses your private key alias and signature algorithm; you choose whether to include the content in the signed data.
Signer	Simple Signer	Creates a simple digital signature with a private key.	Signs the message and places the signature value in a message header, which the receiver can check. Used for lightweight signing.
Signer	XML Digital Signer	Creates an XML signature (enveloped, enveloping or detached).	You select the XML elements to sign with XPath, the canonicalization and the key alias; the signature element is added to the XML.
Verifier	PKCS7 Signature Verifier	Checks a PKCS#7 / CMS signature.	Uses the public key (certificate) of the signer from the keystore; fails the message if the signature is not valid.
Verifier	XML Signature Verifier	Checks an XML digital signature.	Uses the signer's public key or certificate; confirms that the signed elements were not changed.

10. Persistence

Persistence steps store data so it can be used later: either whole messages (Data Store) or single values (variables), or a copy for monitoring.

DATA STORE - whole messages / payloads



VARIABLES - single values that survive between runs



Figure 14 - Data Store operations and variables that survive between iFlow runs.

10.1 Data Store Operations

The data store is a temporary storage in the tenant. Entries are identified by an **Entry ID** inside a named data store, and they expire after a retention period (a default is applied, and it can be configured).

Operation	Definition	Why / how it works
Write	Stores the message (or part of it) as an entry.	You give the data store name, an entry ID (often from a header or XPath), the visibility (this iFlow or global) and retention settings. Can also encrypt the stored data. Typical use: keep data until another iFlow or run picks it up.
Get	Reads one entry by its ID.	Can delete the entry after reading ("Delete on completion"). If the entry does not exist you can choose to raise an error.
Select	Reads several entries at once.	Reads a number of entries (you set how many) and returns them as one message; used for batch processing.
Delete	Removes an entry by its ID.	Cleans up data that is no longer needed.

10.2 Persist

Item	Definition	Why / how it works
Message (Persist Message)	Saves a copy of the message at that point.	The saved payload can be viewed in monitoring, which helps with troubleshooting and auditing. It is not meant as a business store.
Write Variables	Stores named values that survive across separate runs.	You set name, value (constant, header, property, XPath and others), and scope (local to the iFlow or global). Later runs read them with a Content Modifier using the Local Variable or Global Variable source type. Typical use: the "last successful run" timestamp for delta loads.

11. Validator

A **validator** checks that a message follows its schema before processing continues, so errors are found early and clearly.

Item	Definition	Why / how it works
XML Validator	Validates an XML message against an XSD.	You choose the XSD; if the message is not valid an exception is raised and the Exception Subprocess can respond. An option can prevent the exception when validation fails.
EDI Validator	Validates an EDI message against its schema.	Checks structure and content against the EDI schema (the schema comes from the B2B content). Part of the B2B capabilities.

12. Toolbar actions

Item	Definition	Why / how it works
Run Simulation	Runs part of the iFlow inside the designer, without deploying.	You choose a start point and an end point, supply an input message, and the editor shows the message after each step. Good for testing mappings and scripts quickly.
Clear Simulation	Removes the simulation markers and results from the canvas.	Use it when you finish testing or want to start a new simulation.
Help	Opens the documentation for the iFlow editor.	A quick link to the guides for the step or adapter you are working on.

13. Bottom properties (channel configuration)

When you click a channel or a step, its properties appear in the panel at the bottom of the editor. For the sender and receiver channels these hold the adapter settings.

13.1 Sender adapter (HTTPS example)

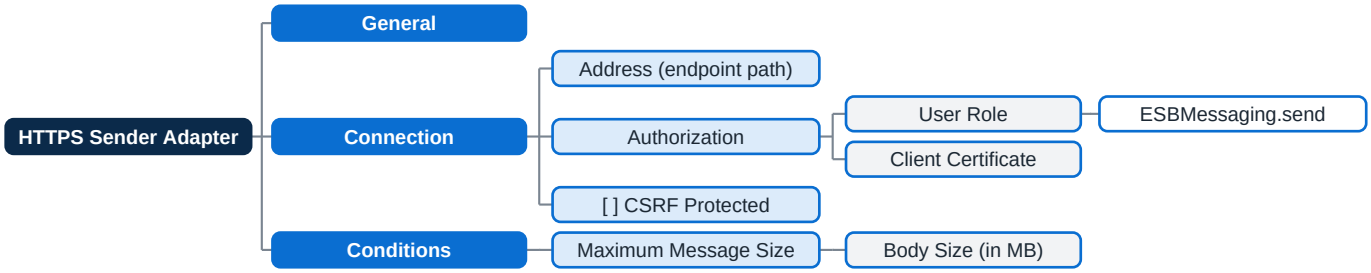


Figure 15 - Property structure of the sender channel.

Field	Definition	Why / how it works
General	Name and description of the channel, and the adapter type.	Identification only.
Connection > Request Processing	Section of the Connection tab with the address and authorization.	Defines how the sender reaches and is allowed to call the iFlow.
Address	The endpoint path suffix of this iFlow.	The call URL is built from the runtime host plus the path, for example https://<runtime-host>/http/<address>. The sender calls this URL.
Authorization	The method the caller is checked with.	**User Role** or **Client Certificate**.
User Role	Caller must have a given role.	The standard role is **ESBMessaging.send**, which allows sending messages to the runtime. Custom roles can be used when you need finer control.
Client Certificate	Caller is identified by a certificate.	The certificate subject or issuer is mapped to a user and role, so mutual TLS can be used.
CSRF Protected (checkbox)	Requires a CSRF token for changing calls.	When on, the caller must first fetch a token (header X-CSRF-Token: Fetch) and send it on the actual call. It protects against cross-site request forgery. Switch it off for system-to-system calls that cannot fetch a token.
Conditions	Limits for incoming messages.	Protects the runtime from messages that are too large.
Maximum message size / Body Size (in MB)	Largest accepted body.	Messages above the limit are rejected with an error, which protects memory and performance.

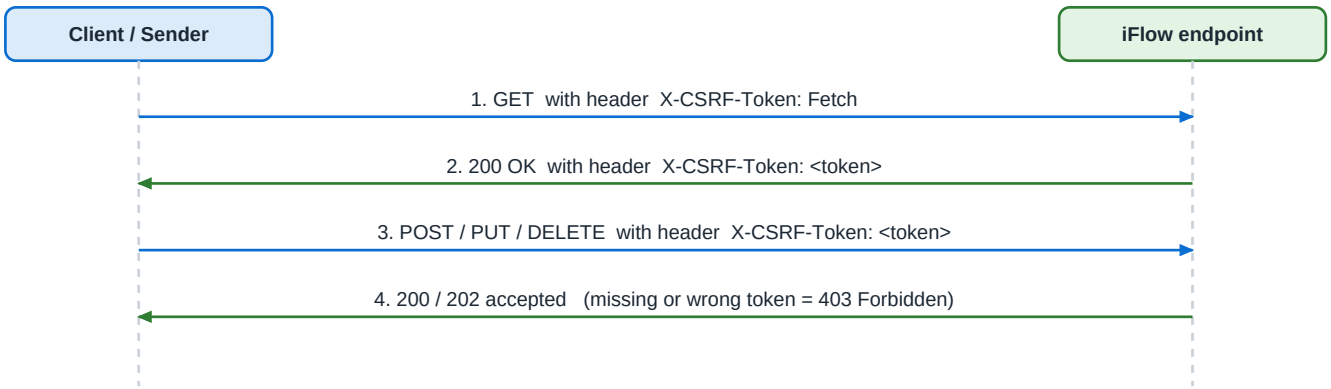


Figure 16 - How CSRF protection works: fetch a token first, then send it with the changing call.

13.2 Receiver adapter (OData example)

The Receiver channel has General, Connection and Processing tabs. The Connection tab holds the details for reaching and authenticating against the target.

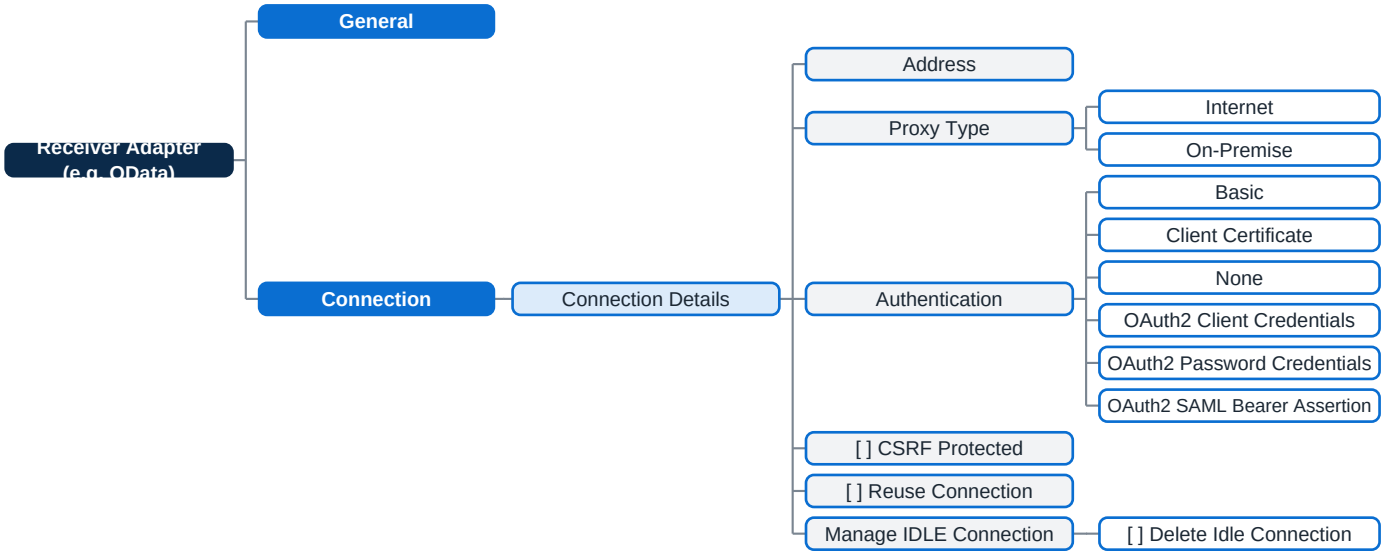


Figure 17 - Connection tab of a receiver channel.

Field	Definition	Why / how it works
General	Name, description and adapter type.	Identification only.
Connection Details	Section with the technical connection settings.	Everything needed to reach and log in to the target.
Address	The URL of the target service.	For OData: the service root, for example https://host/sap/opu/odata/sap/ZSERVICE. Use an externalized parameter so it differs per system.
Proxy Type	How the target is reached.	**Internet**: direct call to a cloud or public endpoint. **On-Premise**: the call goes through the SAP Cloud Connector to a system behind your firewall (a location ID can be set).
Reuse Connection (checkbox)	Keeps connections open for the next call.	Avoids opening a new TCP/TLS connection for each call; improves performance and reduces load.
Manage IDLE Connection / Delete Idle Connection (checkbox)	Controls connections that are open but unused.	Closes idle connections after a time so the runtime does not try to use a connection the target has already dropped (a common cause of connection reset errors).
CSRF Protected (checkbox)	Fetches a CSRF token before changing calls.	The adapter first reads a token from the service and sends it with POST, PUT, MERGE or DELETE. Needed for most SAP Gateway services.

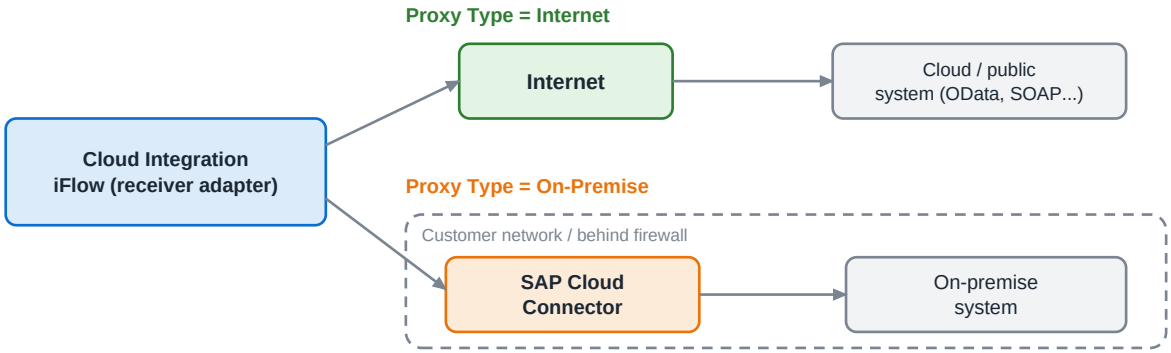


Figure 18 - Proxy Type: Internet versus On-Premise through the Cloud Connector.

Authentication options

Option	How it works	Use when
Basic	Sends user name and password from a stored credential alias.	Simple technical-user access; always over HTTPS.
Client Certificate	The runtime presents a certificate (mutual TLS) from the keystore.	The target requires certificate-based login.

Option	How it works	Use when
None	No credentials are sent.	Public or already protected endpoints (for example by network).
OAuth2 Client Credentials	The adapter gets an access token with client ID and secret (no user).	System-to-system access to modern APIs.
OAuth2 Password Credentials	Token is requested with a user name, password, client ID and secret.	The target supports this grant and a technical user is acceptable.
OAuth2 SAML Bearer Assertion	A SAML assertion is exchanged for an access token.	Principal propagation to SAP systems; the user identity is carried through.

Processing tab

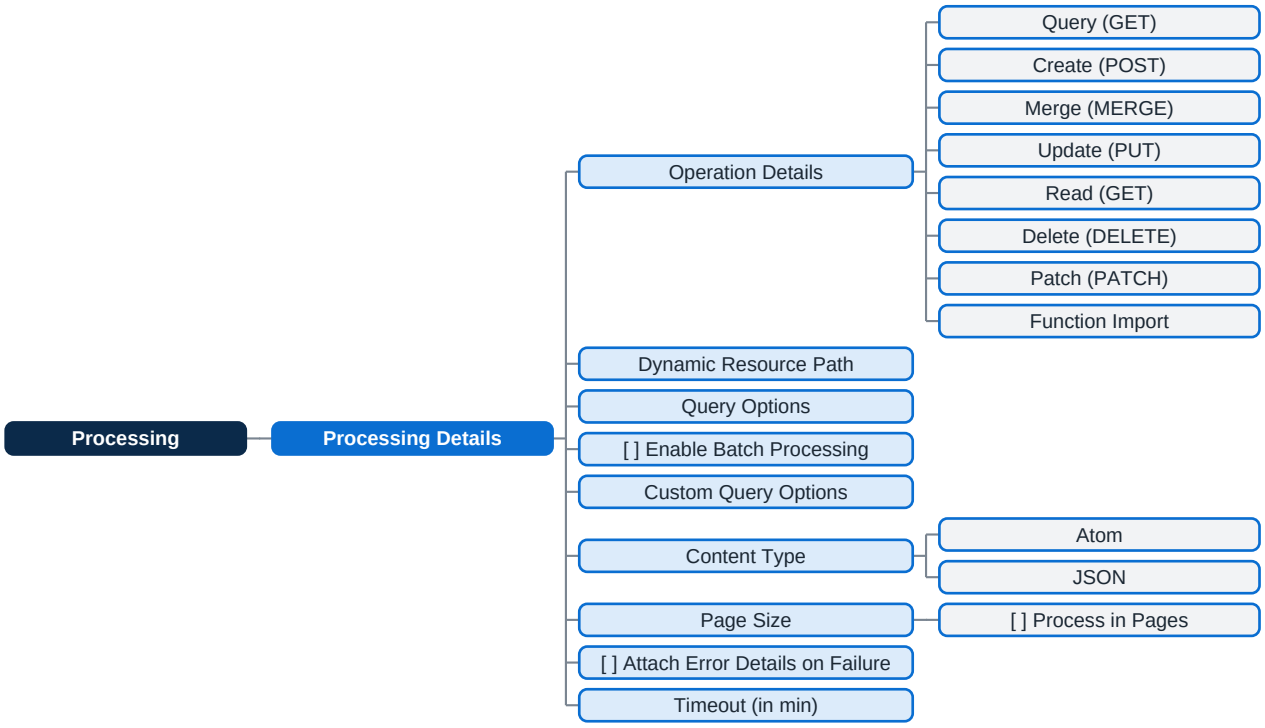


Figure 19 - Processing tab: operation, resource path, query options, content type, paging and timeout.

Operation Details says what the adapter does on the target. The listed operations map to HTTP methods:

Operation	HTTP method	Meaning	Notes
Query	**GET**	Reads a collection of entities, optionally filtered.	Use Query Options such as $filter, $select, $top.
Read	**GET**	Reads one entity by its key.	Key is part of the resource path.
Create	**POST**	Creates a new entity.	The body holds the new entity.
Update	**PUT**	Replaces the whole entity.	Fields not sent can be cleared.
Merge	**MERGE**	Updates only the fields you send.	OData V2 style partial update.
Patch	**PATCH**	Partial update of an entity.	Used where the adapter version offers PATCH.
Delete	**DELETE**	Deletes an entity.	Key is in the resource path.
Function Import	GET / POST	Calls a function defined in the service.	Parameters are passed as query or body.

Field	Definition	Why / how it works
Dynamic Resource Path	The entity path, built at runtime.	You can place header or property values in the path, for example Orders(${header.orderId}), so the same iFlow reads or updates different entities.

Field	Definition	Why / how it works
Query Options	OData system query options.	For example $select=Name,Price&$filter=Status eq 'OPEN'&$top=100. Reduces data and load.
Enable Batch Processing (checkbox)	Sends several operations in one HTTP request.	Reduces round trips and runtime for many small calls.
Custom Query Options	Extra URL parameters that are not OData system options.	For example sap-client=100 or a service-specific parameter, written as name=value pairs.
Content Type	Format of the request and response.	**Atom** (XML) or **JSON**. JSON is lighter and more common.
Page Size / Process in Pages (checkbox)	Reads a large result in chunks.	Instead of loading everything at once, the adapter fetches pages of the given size and passes each page on. Keeps memory use low.
Attach Error Details on Failure (checkbox)	Adds the target's error message to the failure.	The text from the backend is added to the exception, which makes monitoring easier to read.
Timeout (in min)	How long to wait for the response.	The call fails if the target does not answer within this time.

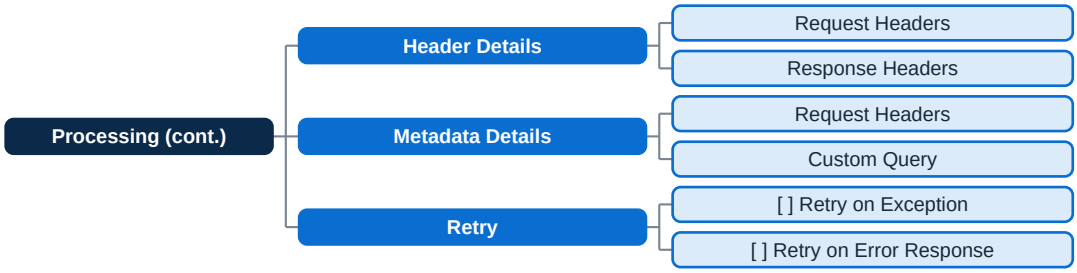


Figure 20 - Header details, metadata details and retry settings.

Field	Definition	Why / how it works
Header details > Request headers	Which message headers are sent to the target.	Header names (or patterns) that are allowed to go out with the HTTP request; others are not passed on.
Header details > Response headers	Which response headers are kept.	Headers from the target's response to copy into the message, for later steps.
Metadata details > Request headers	Headers for the metadata (service description) call.	The adapter reads the service metadata first; these headers go with that call.
Metadata details > Custom Query	Extra URL parameters for the metadata call.	For example sap-client for the $metadata request.
Retry on Exception (checkbox)	Repeats the call when a technical exception occurs.	Covers timeouts and connection errors. The call is made again after a wait.
Retry on Error response (checkbox)	Repeats the call when the target answers with an error status.	Covers server errors such as HTTP 5xx or temporary failures. Do not use it blindly for calls that are not safe to repeat (for example a Create without a duplicate check).

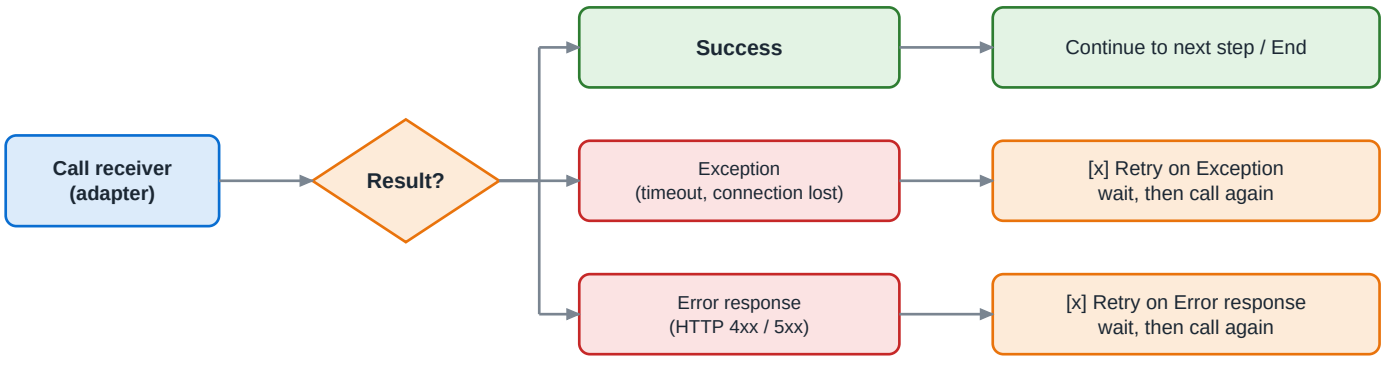


Figure 21 - Retry behaviour after an exception or an error response.

Warning: retrying a POST (Create) can create duplicates when the first attempt actually succeeded but the answer was lost. Use an idempotency key, a Message Digest or a check read before enabling retries on Create.

14. iFlow end to end: Start, Sender, End, Receiver

Putting the items together, an iFlow is read from left to right: **Sender** to **Start**, the steps in the middle, **End** and then the **Receiver**.

Example: JSON request from a client creates an entity in an OData service

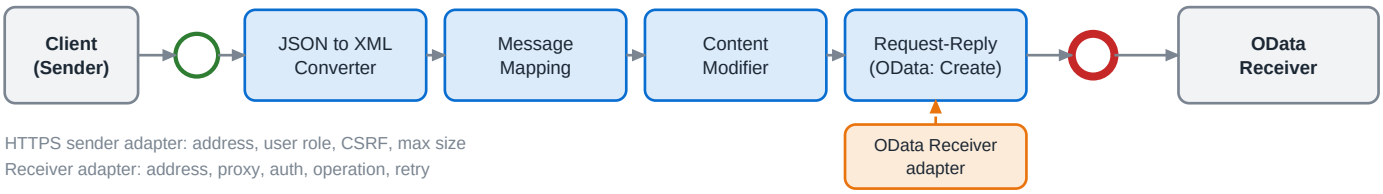


Figure 22 - A typical iFlow: JSON in, map, enrich the headers, create an entity in OData.

No.	What happens	Palette item / property involved
1	The sender calls the endpoint; address, authorization, CSRF token and size limit are checked.	Sender, sender adapter properties
2	The Start Message event begins the Integration Process with the incoming message.	Start Message
3	The body is converted to XML so the rest of the flow can work with it.	JSON to XML Converter
4	Fields are mapped to the structure the target expects.	Message Mapping
5	Headers and properties are set (for example the entity key, content type).	Content Modifier
6	The receiver is called: connection, proxy, authentication, operation, query options, retry.	Request-Reply / End Message with receiver adapter properties
7	The End event finishes the process; in synchronous scenarios the response goes back to the sender.	End Event or End Message
8	On any error the Exception Subprocess runs and logs, alerts or returns a custom error.	Error Start Event, Exception Subprocess, Error End Event

15. Quick reference and best practices

15.1 Which step for which need?

I need to...	Use
Set or change a header, property or body	Content Modifier
Change field structure from source to target	Message Mapping (or XSLT Mapping for complex XML)
Convert between JSON, CSV, EDI and XML	Converters
Take a branch depending on a value	Router (with a Default Route)
Do two things at once with the same message	Parallel Multicast, then Gather or Join
Process each record of a list	General or Iterating Splitter, then Gather
Collect messages arriving over time	Aggregator
Reuse steps or loop	Local Integration Process with Process Call / Looping Process Call
Look up data from another system mid-flow	External Call: Request-Reply or Content Enricher
Keep data until a later run	Data Store Write / Get / Select / Delete
Remember a timestamp between runs	Persist > Write Variables
Protect confidentiality	Encryptor and Decryptor
Prove origin and integrity	Signer and Verifier
Reject invalid XML early	XML Validator
Handle errors	Exception Subprocess with Error Start, log / mail, Error End
Custom logic	Groovy Script or JavaScript

15.2 Best practices

- Convert to XML early and back to the target format late.
- Always give a Router a **Default Route**.
- Add an **Exception Subprocess** to every productive iFlow.
- Externalize addresses, credentials and other environment-specific values.
- Keep the maximum message size and the receiver timeout realistic for the scenario.
- Use Split and Gather (or paging) for large data instead of loading everything at once.
- Do not log or persist sensitive payloads without need; use encryption where required.
- Test with **Run Simulation** first, then deploy and check monitoring.

15.3 Common errors and where to look

Symptom	Likely cause	Check
401 / 403 on the sender call	Missing ESBMessaging.send role, wrong certificate mapping, or CSRF token missing	Sender adapter Authorization, CSRF Protected, caller's role
413 or size error on the sender	Body larger than the configured maximum	Conditions > Body Size (in MB)
Connection reset / stale connection at receiver	Idle connection closed by the target	Reuse Connection, Manage IDLE Connection, Retry on Exception
Cannot reach on-premise system	Cloud Connector, location ID or Proxy Type set wrongly	Proxy Type On-Premise, Cloud Connector mapping
Mapping gives empty target	Wrong namespace, XPath or missing XML conversion	Run Simulation, check namespaces and converter output
Receiver returns 5xx once, then works	Temporary target problem	Retry on Error response with caution